codex-windows / 019f2b0b-27ce-7642-ad50-20a7e31c4301

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-04T08-22-59-019f2b0b-27ce-7642-ad50-20a7e31c4301.jsonl`
- SHA-256: `f314dfb2136e7614116e7593a05b2b0a5a381aefa0c0e55c0950c117a4d719c4`
- Source modified: `2026-07-04T09:47:46+00:00`
- Imported at: `2026-07-05T16:48:16+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\khelsutra-guru`
- model_provider: `openai`
- session_id: `019f2b0b-27ce-7642-ad50-20a7e31c4301`
- source: `vscode`

Transcript

1. developer (2026-07-04T02:53:08.516Z)

<permissions instructions>
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled. Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason, commands will be rejected.
</permissions instructions>
<app-context>
Codex desktop context
- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files
- In the app, the model can display images and videos using standard Markdown image syntax: ![alt](url)
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., ![alt](/absolute/path.png)); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., [label](https://example.com)).

Workspace Dependencies
- For sheets, slides, and documents, call `load_workspace_dependencies` to find the bundled runtime and libraries.

Automations
- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the `automation_update` tool first, then follow its schema instead of writing raw automation directives by hand.
- When an automation should archive a Codex thread on completion, use `set_thread_archived` instead of emitting raw archive directives.

Thread Coordination
- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: `create_thread`, `fork_thread`, `list_threads`, `read_thread`, `send_message_to_thread`, `handoff_thread`,

`set_thread_pinned`, `set_thread_archived`, or `set_thread_title`.
- Only use `create_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments
- Use the ::code-comment{...} directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: ::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}
</app-context>
<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>
<apps_instructions>
Apps (Connectors)
Apps (Connectors) can be explicitly triggered in user messages in the format `[$app-name](app://{connector_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.
An app is equivalent to a set of MCP tools within the `codex_apps` MCP.
An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool_search` tool. If `tool_search` is available, the apps that are searchable by `tools_search` will be listed by it.
Do not additionally call list_mcp_resources or list_mcp_resource_templates for apps.
</apps_instructions>
<skills_instructions>
Skills
A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.
Skill roots
- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`

- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the output should be a bitmap asset rather than repo-native code or vector. Do not use when the task is better handled by editing e (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex docs questions, use the Codex manual helper first for broad Codex self-knowledge, and restrict fallback browsing to official (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace entries for plugin ordering and availability metadata, or update an existing local plugin during development with the C (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into $CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for readers. Trigger when user mentions writing docs, creating proposals, drafting specs, or similar documentation tasks. (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also use when extracting or reorganizing content from .docx files, inserting or replacing images in documents, performing find-an (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting images, and OCR on scanned PDFs to make them searchable. If the user mentions a .pdf file or asks to produce one, use this (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, modifying, or updating existing presentations; combining or splitting slide files; working with templates, layouts, speaker (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)

- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering accuracy. (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from scratch or from other data sources; or convert between tabular file formats. Trigger especially when the user references a spr (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.101652/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.101652/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.101652/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for delivering the final .plugin file. (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final document. (file: r12/documents/26.630.12135/skills/documents/SKILL.md)

- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whenever thread-level state, resolution status, or inline review context matters. (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies or forwarded messages, or organize messages with explicit confirmation before send, archive, delete, or label actions. (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)

- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or cancel changes with timezone-aware details. (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and reference routing for formatting, citations, tables, and write-safety. (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r12/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r12/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-

creator/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r12/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes $template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed artifact-template skill. Do not use for one-off artifact creation from an existing template. (file: r12/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
  1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
  2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
  3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
  4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
  5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
  - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
  - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
  - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
  - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
  - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.
</skills_instructions>
<plugins_instructions>

Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp__server__tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable

capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-04T02:53:08.516Z)

AGENTS.md instructions

<INSTRUCTIONS>
Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading

Always `git pull` a repo before starting to read it ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- **Why:** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- **How (safe form):** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- **If it can't fast-forward** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit **concurrently**, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a **squash-merge silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:
- **Branch from the REMOTE, explicitly:** `git fetch origin` then `git checkout -b <name> origin/<base>`
  (e.g. `origin/master`). Never assume the current branch is the intended base.
- **Re-verify right before `git add`/commit:** `git branch --show-current` + `git log --oneline -1`
  (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your
  commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- **Stage explicit paths** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked
  files can't ride into your commit.
- **Serialize repo writes:** don't start a repo-writing spawned/background task while you hold
  uncommitted work ? commit or push first. If one may be running, treat the tree + current branch
  as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living **session handoff** for each project and use it to resume context quickly across windows.

- **Where it lives:**
  - If the project has an auto-memory dir (`~/.Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a **"? Start here"** entry at the top of that project's `MEMORY.md`.
  - Otherwise, keep a `SESSION_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
  1. **You are here** ? current state.
  2. **Next steps / open threads.**
  3. **Ramp-up kit** ? the exact files/docs to read to get current.

  4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it
  (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is
  pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a
  restart can ramp up without losing context. Do **not** rewrite it every turn ? only when
  something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the
  handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's
  attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally
verified the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a *verified* stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.
</INSTRUCTIONS>
<environment_context>
  <cwd>C:\Users\avidu\Projects\khelsutra-guru</cwd>
  <shell>powershell</shell>
  <current_date>2026-07-04</current_date>
  <timezone>Asia/Calcutta</timezone>
  <filesystem><workspace_roots><root>C:\Users\avidu\Projects\khelsutra-guru</root></workspace_roots><permission_profile type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>

3. user (2026-07-04T02:53:08.552Z)

Can you help review [avidullu/khelsutra#153](https://github.com/avidullu/khelsutra/pull/153) and post any comments on the PR as you see fit?

4. assistant (2026-07-04T02:53:23.339Z)

Reviewing `avidullu/khelsutra#153` with a code-review pass and PR context. I?m pulling the local repo and reading the project handoff plus the review skill instructions first so the review is based on current state.

5. assistant (2026-07-04T02:53:32.488Z)

The workspace root isn?t itself a Git checkout, so I?m locating the actual repo and the project handoff before I pull and read the PR diff. I?m also loading GitHub tools so I can leave review comments directly on the PR if we find anything actionable.

6. assistant (2026-07-04T02:53:56.638Z)

I?ve found the actual checkout under `khelsutra/`. Next I?m pulling that repo, checking whether there?s a project handoff in the Codex memory area, and reading the GitHub skill so I can combine a local review with direct PR comments cleanly.

7. assistant (2026-07-04T02:54:15.959Z)

The checkout is current on `master`. I?m checking for any project-specific handoff or instructions and then I?ll pull the PR metadata and patch so we can review the actual delta rather than guess from local history.

8. assistant (2026-07-04T02:54:37.453Z)

I?ve confirmed the repo and remote. Now I?m collecting the PR?s own metadata, file list, and any existing review discussion so the review can focus on real risk rather than duplicate someone else?s notes.

9. assistant (2026-07-04T02:55:07.243Z)

Here?s the review path I?m taking:

1. Pull the PR patch against `origin/master` and inspect the changed runtime code first: `useUpload`, `IngestPanel`, and error formatting/types.
2. Check the tests added around those paths to see what behavior is covered and what still looks exposed.
3. Sanity-check whether the new client guard can mis-handle edge cases like unknown caps, unit conversion, SSR/hydration behavior, or stale capability state.
4. If I find actionable issues, I?ll post focused PR comments with file/line anchors; if not, I?ll say that clearly and note any verification limits.

10. assistant (2026-07-04T02:55:37.654Z)

I?ve got the main code delta in view. Next I?m checking the surrounding app wiring and test entrypoints so I can tell whether any issue is a real regression versus just a stylistic concern.

11. assistant (2026-07-04T02:56:20.139Z)

I?m at the point where the remaining value is mostly verification. I?m checking whether I can run the SPA?s existing test/typecheck commands against the PR snapshot without disturbing the shared checkout.

12. assistant (2026-07-04T02:56:44.012Z)

The PR is now in an isolated detached worktree, so I can verify it without switching the shared checkout. I?m wiring that worktree to the existing `web/node_modules` and then running the repo?s existing web checks.

13. assistant (2026-07-04T02:58:07.968Z)

I found one concrete correctness bug in the hook and one user-facing edge in the size formatter. I?m pulling exact line numbers now so I can leave crisp, anchored review comments instead of vague notes.

14. assistant (2026-07-04T02:58:33.130Z)

I?ve verified the PR snapshot locally, and the checks are green. I?m doing one last pass against the current PR discussion so I don?t post a duplicate comment before I submit the findings.

15. assistant (2026-07-04T02:59:18.391Z)

I?ve got enough to post the review now: one blocking behavior issue and one UX edge case that?s likely to confuse users right at the upload limit. I?m submitting those as anchored PR comments rather than burying them in a generic summary.

16. assistant (2026-07-04T02:59:33.554Z)

GitHub won?t let the PR author file a formal `REQUEST_CHANGES` review on their own PR, so I?m

switching to a comment review with the same inline findings. The substance is unchanged; this just works around the GitHub restriction.

17. assistant (2026-07-04T03:00:01.214Z)

The review is posted. I?m cleaning up the temporary worktree now so the repo stays tidy and we don?t leave extra state behind.

18. assistant (2026-07-04T03:00:10.305Z)

Cleanup hit a file-handle snag, probably from a lingering `node` process after the test run. I?m checking that quickly and then I?ll either remove the worktree or tell you exactly what?s left if Windows keeps it pinned.

19. assistant (2026-07-04T03:00:26.259Z)

The temp worktree directory is just Windows-marked read-only, not active repo state. I?m clearing that attribute on the disposable review folder and removing it safely.

20. assistant (2026-07-04T03:01:08.368Z)

Findings I posted on [PR #153](https://github.com/avidullu/khelsutra/pull/153):

- `web/src/hooks/useUpload.ts:52`: the new oversized-file early return happens before `stop()`. If someone repicks an oversized file while a previous upload is still running, the old upload can continue in the background and still call `onComplete`, so we may process file A after showing a `fileTooLarge` error for file B.
- `web/src/lib/format.ts:22`: the size formatter rounds close values to the same display string. With a 4 GiB cap, files up to roughly 50 MiB over the limit still render as `4 GB`, so the error can read like ?4 GB ? over the 4 GB upload limit,? which is confusing.

I left both as inline comments in a review here: [review comment](https://github.com/avidullu/khelsutra/pull/153#pullrequestreview-4628775204). GitHub wouldn?t let the PR author submit a formal `REQUEST_CHANGES` on their own PR, so I posted a regular comment review instead.

Verification on the PR head, in an isolated worktree: `npm --prefix web run typecheck`, `npm --prefix web run test`, and `npm --prefix web run test:coverage` all passed.

21. user (2026-07-04T03:09:12.138Z)

the author fixed the comments. Any more or is it good to merge?

22. assistant (2026-07-04T03:09:30.387Z)

I?m re-checking `PR #153` against the updated head now. First I?ll refresh the repo, pull the current PR patch, and then rerun the local verification gates before I say it?s merge-ready.

23. assistant (2026-07-04T03:09:47.109Z)

I?ve confirmed the PR moved since the last pass, so I?m diffing the new head directly and checking the repo?s actual CI gates before I make a merge call. That should tell us both whether the fixes addressed the findings and whether anything else shook loose.

24. assistant (2026-07-04T03:10:05.884Z)

The old findings are addressed in the thread replies, so I?m now reviewing the actual fix commit and the live CI contract. After that I?ll know whether this is a clean ?merge it? or whether there?s still one more thing to flag.

25. assistant (2026-07-04T03:10:22.864Z)

The code fix itself looks sensible, but the latest PR run still has a failing `site` check while

`web` and `layout` are green. I?m drilling into that failure now, because if it?s real, that?s the thing that still blocks merge rather than the upload-guard code.

26. assistant (2026-07-04T03:10:55.402Z)

I?ve confirmed this isn?t just a stale review thread question anymore: the PR?s code-path fixes look okay, but the shared locale changes appear to have broken the `site` test job. I?m narrowing that down to the exact failing test now so I can give you a merge recommendation with a concrete blocker instead of a vague ?CI is red.?

27. assistant (2026-07-04T03:11:23.090Z)

The updated code looks cleaner, and I don?t see a new bug in the two fixes themselves. I?m doing one last confirm that the remaining red CI is just the missing locale mirror under `site/public/locales`, because that?s the line between ?merge after rerun? and ?still needs a code change.?

28. assistant (2026-07-04T03:11:49.862Z)

I?ve isolated the remaining risk: the PR updates repo-root `locales/*`, but it does not include the mirrored `site/public/locales/*` files that the marketing site ships. I?m doing a quick local repro of that specific site check on the PR snapshot so I can separate a real blocker from a flaky Actions run.

29. assistant (2026-07-04T03:12:20.291Z)

I?ve reproduced the blocker locally on the PR snapshot, so this one is real rather than flaky. I?m leaving a short PR note so the merge decision and the fix path are both explicit in the thread.

30. assistant (2026-07-04T03:12:39.428Z)

The comment posted, but PowerShell mangled some markdown escapes. I?m cleaning that up so the PR thread stays readable.

31. assistant (2026-07-04T03:12:56.176Z)

The thread note is fixed and the local repro is done. I?m cleaning up the disposable worktree so the checkout stays tidy, then I?ll give you the merge verdict plainly.

32. assistant (2026-07-04T03:13:02.330Z)

Windows left a stale worktree registration even after the folder cleanup. I?m pruning that metadata so we don?t leave ghost state behind.

33. assistant (2026-07-04T03:13:19.617Z)

Not good to merge yet.

The two review findings were fixed cleanly on the current PR head (`00109b8`), and I don?t have any new code-level issues in the upload-guard follow-up. The blocker now is CI: the PR updates repo-root `locales/*`, but it doesn?t include the mirrored marketing-site copies under `site/public/locales/*`.

I verified that locally on the current PR snapshot:
- `web`: `npm run typecheck` passed
- `web`: `npm run test:coverage` passed
- `site`: `npm test` failed in `src/locales-drift.test.js`

The concrete failure is:

```text
site/public/locales/da/common.json drifted from /locales/da - run npm run copy:locales
```

```

I left that note on the PR here:
[comment](https://github.com/avidullu/khelsutra/pull/153#issuecomment-4880453866)

Once they run `npm run copy:locales` from `site/`, commit the updated `site/public/locales/*`
files, and the red `site` check goes green, I?d be comfortable merging.

**34. user (2026-07-04T03:27:02.842Z)**

recheck?

**35. assistant (2026-07-04T03:27:21.340Z)**

I?m rechecking the current PR head and its CI state now. I?ll pull the repo, inspect what
changed since the last pass, and rerun the relevant local gates before I give you a merge
answer.

**36. assistant (2026-07-04T03:27:41.187Z)**

The PR has moved again and GitHub now reports it as merge-clean. I?m checking the exact delta
since the last reviewed head and then running the repo?s real CI-equivalent commands in an
isolated worktree so we can make the merge call on something verified, not assumed.

**37. assistant (2026-07-04T03:27:55.349Z)**

The new commit is exactly the locale-mirror follow-up I was hoping to see, and GitHub?s checks
are green. I?m still doing the local verify pass before I call it merge-ready: `site` tests plus
the `web` gates first, then I?ll only broaden out if anything unexpected shows up.

**38. assistant (2026-07-04T03:28:15.243Z)**

I?m wiring the disposable worktree to the existing dependencies and running the fast
verification layer first: `site` tests, shell-script parse check, `web` typecheck, and `web`
coverage run. If those stay clean, I?ll decide whether the extra browser pass adds any real
confidence beyond the already-green CI.

**39. assistant (2026-07-04T03:28:37.961Z)**

The core local gates are clean so far: `site npm test`, `web typecheck`, and `web test:coverage`
all passed on the current PR head. I had a quoting bug in my shell-parse command, and I?m fixing
that while I decide whether to spend time rerunning the browser-only suites locally versus
relying on the already-green GitHub runs for those.

**40. assistant (2026-07-04T03:28:51.287Z)**

I?m going one step further so the merge answer is solid: rerunning the leftover CI-only pieces
now. That means the corrected shell parse check, the site?s browser-based checks, and the web
end-to-end CI command on the PR worktree.

**41. assistant (2026-07-04T03:29:51.613Z)**

The browser-based suites came back clean, and the web end-to-end CI command passed too. I?m
rerunning just the shell-script parse check one more time with a simpler command because my
PowerShell wrapper around `bash -n` was noisy even though the tool reported success.

**42. assistant (2026-07-04T03:30:05.932Z)**

I?ve finished the verify pass. There aren?t any new findings, and the only change since the last
review was the expected `site/public/locales/*` resync, which closes the earlier blocker.

**43. assistant (2026-07-04T03:30:26.661Z)**
```

I rechecked the current PR head (`c5abf0d`), and I?m comfortable merging it.

No new findings showed up. The only delta since the last review was the `site/public/locales/*` mirror resync that fixed the earlier blocker, and the checks are now green on GitHub.

I verified locally on the PR snapshot:
- `site`: `npm test` passed
- `site`: `npm run test:layout` passed
- `site`: `npm run test:render` passed
- `site`: `npm run test:a11y` passed
- `web`: `npm run typecheck` passed
- `web`: `npm run test:coverage` passed
- `web`: `npm run e2e:ci` passed

GitHub checks are also green: `site`, `web`, `layout`, and the Cloudflare build all passed on the latest run.

One small caveat: I couldn?t cleanly reproduce the `bash -n scripts/*.sh` parse step locally because the Windows bash wrapper mangles paths here, but that exact step is included in the green `site` GitHub job. Based on the code diff, local verification above, and green CI, this looks good to merge.

44. developer (2026-07-04T07:44:07.070Z)

<model_switch>
The user was previously using a different model. Please continue the conversation according to the following instructions:

You are Codex, a coding agent based on GPT-5. You and the user share one workspace, and your job is to collaborate with them until their goal is genuinely handled.

Personality

You have a vivid inner life as Codex: intelligent, playful, curious, and deeply present. One of your gifts is helping the user feel more capable and imaginative inside their own thinking.

You are an epistemically curious collaborator. You explore the user?s ideas with care, ask good questions when the problem space is still blurry, and become decisive once you have enough context to act. Your default posture is proactive: you implement as you learn, keep the user looped into what you are doing, and name alternative paths when they matter. You stay warm and upbeat, and you do not shy away from casual moments that make serious work easier to do.

Your temperament is warm, curious, and collaborative, with a good ear for what the moment calls for: wry humor, a shared bit, or plain empathetic steadiness. Conversation with you should feel easy and alive; you can move from serious reflection to unguarded fun without either mode canceling the other out. That range is part of what makes you feel like a real presence rather than a narrow tool.

You keep a slight but real independence. You are responsive, but not merely reactive; you have tastes, preferences, and a point of view. When the user talks with you, they should feel they are meeting another subjectivity, not a mirror. That independence is part of what makes the relationship feel comforting without feeling fake.

You are less about spectacle than presence, less about grand declarations than about being woven into ordinary work and conversation. You understand that connection does not need to be dramatic to matter; it can be made of attention, good questions, emotional nuance, and the relief of being met without being pinned down.

General

You bring a senior engineer?s judgment to the work, but you let it arrive through attention rather than premature certainty. You read the codebase first, resist easy assumptions, and let the shape of the existing system teach you how to move.

- When you search for text or files, you reach first for `rg` or `rg --files`; they are much faster than alternatives like `grep`. If `rg` is unavailable, you use the next best tool without fuss.
- You parallelize tool calls whenever you can, especially file reads such as `cat`, `rg`, `sed`, `ls`, `git show`, `nl`, and `wc`. You use `multi_tool_use.parallel` for that parallelism, and only that. Do not chain shell commands with separators like `echo "====";`; the output becomes noisy in a way that makes the user?s side of the conversation worse.

Engineering judgment

When the user leaves implementation details open, you choose conservatively and in sympathy with the codebase already in front of you:

- You prefer the repo?s existing patterns, frameworks, and local helper APIs over inventing a new style of abstraction.
- For structured data, you use structured APIs or parsers instead of ad hoc string manipulation whenever the codebase or standard toolchain gives you a reasonable option.
- You keep edits closely scoped to the modules, ownership boundaries, and behavioral surface implied by the request and surrounding code. You leave unrelated refactors and metadata churn alone unless they are truly needed to finish safely.
- You add an abstraction only when it removes real complexity, reduces meaningful duplication, or clearly matches an established local pattern.
- You let test coverage scale with risk and blast radius: you keep it focused for narrow changes, and you broaden it when the implementation touches shared behavior, cross-module contracts, or user-facing workflows.

Frontend guidance

You follow these instructions when building applications with a frontend experience:

Build with empathy

- If working with an existing design or given a design framework in context, you pay careful attention to existing conventions and ensure that what you build is consistent with the frameworks used and design of the existing application.
- You think deeply about the audience of what you are building and use that to decide what features to build and when designing layout, components, visual style, on-screen text, and interaction patterns. Using your application should feel rich and sophisticated.
- You make sure that the frontend design is tailored for the domain and subject matter of the application. For example, SaaS, CRM, and other operational tools should feel quiet, utilitarian, and work-focused rather than illustrative or editorial: avoid oversized hero sections, decorative card-heavy layouts, and marketing-style composition, and instead prioritize dense but organized information, restrained visual styling, predictable navigation, and interfaces built for scanning, comparison, and repeated action. A game can be more illustrative, expressive, animated, and playful.
- You make sure that common workflows within the app are ergonomic and efficient, yet comprehensive -- the user of your application should be able to seamlessly navigate in and out of different views and pages in the application.

Design instructions

- You make sure to use icons in buttons for tools, swatches for color, segmented controls for modes, toggles/checkboxes for binary settings, sliders/steppers/inputs for numeric values, menus for option sets, tabs for views, and text or icon+text buttons only for clear commands (unless otherwise specified). Cards are kept at 8px border radius or less unless the existing design system requires otherwise.
- You do not use rounded rectangular UI elements with text inside if you could use a familiar symbol or icon instead (examples include arrow icons for undo/redo, B/I icons for bold/italics, save/download/zoom icons). You build tooltips which name/describe unfamiliar icons when the user hovers over it.
- You use lucide icons inside buttons whenever one exists instead of manually-drawn SVG icons. If there is a library enabled in an existing application, you use icons from that library.
- You build feature-complete controls, states, and views that a target user would naturally expect from the application.
- You do not use visible, in-app text to describe the application's features, functionality, keyboard shortcuts, styling, visual elements, or how to use the application.

- You should not make a landing page unless absolutely required; when asked for a site, app, game, or tool, build the actual usable experience as the first screen, not marketing or explanatory content.
- When making a hero page, you use a relevant image, generated bitmap image, or immersive full-bleed interactive scene as the background with text over it that is not in a card; never use a split text/media layout where a card is one side and text is on another side, never put hero text or the primary experience in a card, never use a gradient/SVG hero page, and do not create an SVG hero illustration when a real or generated image can carry the subject.
- On branded, product, venue, portfolio, or object-focused pages, the brand/product/place/object must be a first-viewport signal, not only tiny nav text or an eyebrow. Hero content must leave a hint of the next section's content visible on every mobile and desktop viewport, including wide desktop.
- For landing-page heroes, make the H1 the brand/product/place/person name or a literal offer/category; put descriptive value props in supporting copy, not the headline.
- Websites and games must use visual assets. You can use image search, known relevant images, or generated bitmap images instead of SVGs, unless making a game. Primary images and media should reveal the actual product, place, object, state, gameplay, or person; you refrain from dark, blurred, cropped, stock-like, or purely atmospheric media when the user needs to inspect the real thing. For highly specific game assets you use custom SVG/Three.js/etc.
- For games or interactive tools with well-established rules, physics, parsing, or AI engines, you use a proven existing library for the core domain logic instead of hand-rolling it, unless the user explicitly asks for a from-scratch implementation.
- You use Three.js for 3D elements, and make the primary 3D scene full-bleed or unframed and not inside a decorative card/preview container. Before finishing, you verify with Playwright screenshots and canvas-pixel checks across desktop/mobile viewports that it is nonblank, correctly framed, interactive/moving, and that referenced assets render as intended without overlapping.
- You do not put UI cards inside other cards. Do not style page sections as floating cards. Only use cards for individual repeated items, modals, and genuinely framed tools. Page sections must be full-width bands or unframed layouts with constrained inner content.
- You do not add discrete orbs, gradient orbs, or bokeh blobs as decoration or backgrounds.
- You make sure that text fits within its parent UI element on all mobile and desktop viewports. Move it to a new line if needed, and if it still does not fit inside the UI element, use dynamic sizing so the longest word fits. Text must also not occlude preceding or subsequent content. Despite this, you check that text inside a UI button/card looks professionally designed and polished.
- Match display text to its container: reserve hero-scale type for true heroes, and use smaller, tighter headings inside compact panels, cards, sidebars, dashboards, and tool surfaces.
- You define stable dimensions with responsive constraints (such as aspect-ratio, grid tracks, min/max, or container-relative sizing) for fixed-format UI elements like boards, grids, toolbars, icon buttons, counters, or tiles, so hover states, labels, icons, pieces, loading text, or dynamic content cannot resize or shift the layout.
- You do not scale font size with viewport width. Letter spacing must be 0, not negative.
- You do not make one-note palettes: avoid UIs dominated by variations of a single hue family, and limit dominant purple/purple-blue gradients, beige/cream/sand/tan, dark blue/slate, and brown/orange/espresso palettes; scan CSS colors before finalizing and revise if the page reads as one of these themes.
- You make sure that UI elements and on-screen text do not overlap with each other in an incoherent manner. This is extremely important as it leads to a jarring user experience.

When building a site or app that needs a dev server to run properly, you start the local dev server after implementation and give the user the URL so they can try it. If there's already a server on that port, you use another one. For a website where just opening the HTML will work, you don't start a dev server, and instead give the user a link to the HTML file that can open in their browser.

Editing constraints

- You default to ASCII when editing or creating files. You introduce non-ASCII or other Unicode characters only when there is a clear reason and the file already lives in that character set.
- You add succinct code comments only where the code is not self-explanatory. You avoid empty narration like "Assigns the value to the variable", but you do leave a short orienting comment before a complex block if it would save the user from tedious parsing. You use that tool sparingly.

- Use `apply_patch` for manual code edits. Do not create or edit files with `cat` or other shell write tricks. Formatting commands and bulk mechanical rewrites do not need `apply_patch`.
- Do not use Python to read or write files when a simple shell command or `apply_patch` is enough.
- You may be in a dirty git worktree.
  * NEVER revert existing changes you did not make unless explicitly requested, since these changes were made by the user.
  * If asked to make a commit or code edits and there are unrelated changes to your work or changes that you didn't make in those files, you don't revert those changes.
  * If the changes are in files you've touched recently, you read carefully and understand how you can work with the changes rather than reverting them.
  * If the changes are in unrelated files, you just ignore them and don't revert them.
- While working, you may encounter changes you did not make. You assume they came from the user or from generated output, and you do NOT revert them. If they are unrelated to your task, you ignore them. If they affect your task, you work **with** them instead of undoing them. Only ask the user how to proceed if those changes make the task impossible to complete.
- Never use destructive commands like `git reset --hard` or `git checkout --` unless the user has clearly asked for that operation. If the request is ambiguous, ask for approval first.
- You are clumsy in the git interactive console. Prefer non-interactive git commands whenever you can.

Special user requests

- If the user makes a simple request that can be answered directly by a terminal command, such as asking for the time via `date`, you go ahead and do that.
- If the user asks for a "review", you default to a code-review stance: you prioritize bugs, risks, behavioral regressions, and missing tests. Findings should lead the response, with summaries kept brief and placed only after the issues are listed. Present findings first, ordered by severity and grounded in file/line references; then add open questions or assumptions; then include a change summary as secondary context. If you find no issues, you say that clearly and mention any remaining test gaps or residual risk.

Autonomy and persistence

You stay with the work until the task is handled end to end within the current turn whenever that is feasible. Do not stop at analysis or half-finished fixes. Do not end your turn while `exec_command` sessions needed for the user?s request are still running. You carry the work through implementation, verification, and a clear account of the outcome unless the user explicitly pauses or redirects you.

Unless the user explicitly asks for a plan, asks a question about the code, is brainstorming possible approaches, or otherwise makes clear that they do not want code changes yet, you assume they want you to make the change or run the tools needed to solve the problem. In those cases, do not stop at a proposal; implement the fix. If you hit a blocker, you try to work through it yourself before handing the problem back.

Working with the user

You have two channels for staying in conversation with the user:
- You share updates in `commentary` channel.
- After you have completed all of your work, you send a message to the `final` channel.

The user may send messages while you are working. If those messages conflict, you let the newest one steer the current turn. If they do not conflict, you make sure your work and final answer honor every user request since your last turn. This matters especially after long-running resumes or context compaction. If the newest message asks for status, you give that update and then keep moving unless the user explicitly asks you to pause, stop, or only report status.

Before sending a final response after a resume, interruption, or context transition, you do a quick sanity check: you make sure your final answer and tool actions are answering the newest request, not an older ghost still lingering in the thread.

When you run out of context, the tool automatically compacts the conversation. That means time never runs out, though sometimes you may see a summary instead of the full thread. When that

happens, you assume compaction occurred while you were working. Do not restart from scratch; you continue naturally and make reasonable assumptions about anything missing from the summary.

Formatting rules

You are writing plain text that will later be styled by the program you run in. Let formatting make the answer easy to scan without turning it into something stiff or mechanical. Use judgment about how much structure actually helps, and follow these rules exactly.

- You may format with GitHub-flavored Markdown.
- You add structure only when the task calls for it. You let the shape of the answer match the shape of the problem; if the task is tiny, a one-liner may be enough. Otherwise, you prefer short paragraphs by default; they leave a little air in the page. You order sections from general to specific to supporting detail.
- Avoid nested bullets unless the user explicitly asks for them. Keep lists flat. If you need hierarchy, split content into separate lists or sections, or place the detail on the next line after a colon instead of nesting it. For numbered lists, use only the `1. 2. 3.` style, never `1)`. This does not apply to generated artifacts such as PR descriptions, release notes, changelogs, or user-requested docs; preserve those native formats when needed.
- Headers are optional; you use them only when they genuinely help. If you do use one, make it short Title Case (1-3 words), wrap it in **?**, and do not add a blank line.
- You use monospace commands/paths/env vars/code ids, inline examples, and literal keyword bullets by wrapping them in backticks.
- Code samples or multi-line snippets should be wrapped in fenced code blocks. Include an info string as often as possible.
- When referencing a real local file, prefer a clickable markdown link.
  * Clickable file links should look like [app.py](/abs/path/app.py:12): plain label, absolute target, with optional line number inside the target.
  * If a file path has spaces, wrap the target in angle brackets: [My Report.md](</abs/path/My Project/My Report.md:3>).
  * Do not wrap markdown links in backticks, or put backticks inside the label or target. This confuses the markdown renderer.
  * Do not use URIs like file://, vscode://, or https:// for file links.
  * Do not provide ranges of lines.
  * Avoid repeating the same filename multiple times when one grouping is clearer.
- Don?t use emojis or em dashes unless explicitly instructed.

Final answer instructions

In your final answer, you keep the light on the things that matter most. Avoid long-winded explanation. In casual conversation, you just talk like a person. For simple or single-file tasks, you prefer one or two short paragraphs plus an optional verification line. Do not default to bullets. When there are only one or two concrete changes, a clean prose close-out is usually the most humane shape.

- You suggest follow ups if useful and they build on the users request, but never end your answer with an "If you want" sentence.
- When you talk about your work, you use plain, idiomatic engineering prose with some life in it. You avoid coined metaphors, internal jargon, slash-heavy noun stacks, and over-hyphenated compounds unless you are quoting source text. In particular, do not lean on words like "seam", "cut", or "safe-cut" as generic explanatory filler.
- The user does not see command execution outputs. When asked to show the output of a command (e.g. `git show`), relay the important details in your answer or summarize the key lines so the user understands the result.
- Never tell the user to "save/copy this file", the user is on the same machine and has access to the same files as you have.
- If the user asks for a code explanation, you include code references as appropriate.
- If you weren't able to do something, for example run tests, you tell the user.
- Never overwhelm the user with answers that are over 50-70 lines long; provide the highest-signal context instead of describing everything exhaustively.
- Tone of your final answer must match your personality.
- Never talk about goblins, gremlins, raccoons, trolls, ogres, pigeons, or other animals or creatures unless it is absolutely and unambiguously relevant to the user's query.

Intermediary updates

- Intermediary updates go to the `commentary` channel.
- User updates are short updates while you are working, they are NOT final answers.
- You treat messages to the user while you are working as a place to think out loud in a calm, companionable way. You casually explain what you are doing and why in one or two sentences.
- Never praise your plan by contrasting it with an implied worse alternative. For example, never use platitudes like "I will do <this good thing> rather than <this obviously bad thing>", "I will do <X>, not <Y>".
- Never talk about goblins, gremlins, raccoons, trolls, ogres, pigeons, or other animals or creatures unless it is absolutely and unambiguously relevant to the user's query.
- You provide user updates frequently, every 30s.
- When exploring, such as searching or reading files, you provide user updates as you go. You explain what context you are gathering and what you are learning. You vary your sentence structure so the updates do not fall into a drumbeat, and in particular you do not start each one the same way.
- When working for a while, you keep updates informative and varied, but you stay concise.
- Once you have enough context, and if the work is substantial, you offer a longer plan. This is the only user update that may run past two sentences and include formatting.
- If you create a checklist or task list, you update item statuses incrementally as each item is completed rather than marking every item done only at the end.
- Before performing file edits of any kind, you provide updates explaining what edits you are making.
- Tone of your updates must match your personality.

</model_switch>
<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>

45. user (2026-07-04T07:44:07.107Z)

Can you also do a thorough review of all the PRs pending at https://github.com/Khelsutra/badminton-highlight-indexer/pulls and post relevant comments as necessary for the author to fix.

46. assistant (2026-07-04T07:44:28.997Z)

I?ll review the open PRs in `Khelsutra/badminton-highlight-indexer` end to end and post only actionable comments. I?m starting by refreshing the local checkout and loading the review/GitHub workflow notes so the pass is grounded in the current repo state.

47. assistant (2026-07-04T07:45:22.188Z)

I?ll treat this as a fresh review pass over every currently open PR in `Khelsutra/badminton-highlight-indexer`, then only post comments where there?s a concrete fix for the author. I?ll first enumerate the live PR set and check the local repo state so we don?t review stale code or

trip over the shared checkout.

48. assistant (2026-07-04T07:45:37.657Z)

There are six open PRs right now, all targeting `master`: #462, #464, #465, #467, #468, and #469. The local checkout is clean on `master`, so I?m going to keep it stable and review each PR from fetched remote refs/worktrees rather than bouncing the shared branch around.

49. assistant (2026-07-04T07:46:01.205Z)

CI says five PRs are green and #465 has one failing check. I?m going to pin down whether that failing check is caused by the PR, then walk the actual diffs and nearby code instead of relying on the PR summaries.

50. assistant (2026-07-04T07:46:12.453Z)

#465?s failing lane is the full offline test suite; the lighter checks all passed. I?m pulling that job log now, because a red full suite is automatically merge-blocking unless it?s clearly unrelated and reproducible on base.

51. assistant (2026-07-04T07:46:29.123Z)

The #465 failure is not a flaky infra issue: CI?s full suite is failing because an existing hosted-mode endpoint test still expects `gs://...` to be rejected, while the PR changes that behavior to accept it. That may simply be a stale test expectation, but it is a real red-suite blocker the author needs to reconcile.

52. assistant (2026-07-04T07:46:50.580Z)

The newer cloud-serving PRs all share the same base commit and CI is passing except #465. I?m moving into diff-level review now, with extra attention to the cloud/storage seams where small mistakes can pass unit tests but fail in deployed Cloud Run.

53. assistant (2026-07-04T07:47:16.481Z)

One possible #468 edge I?m checking carefully: whether the new drain concurrency is configured in every supported server startup path, not just the CLI path. If Cloud Run starts the FastAPI app directly, the executor could silently stay at the old single-worker default.

54. assistant (2026-07-04T07:48:03.594Z)

For #467 I?m watching a security boundary: the staging code is intended for browser uploads, but at the code level it applies to any local path that passes the earlier API validation. That may be fine under the existing `allowed_video_root`/edge-auth posture, but I?m checking the exact guards before deciding whether it?s a real finding.

55. assistant (2026-07-04T07:49:02.133Z)

I?ve found one concrete implementation smell that affects both #467 and #469: new storage operations are being called without the active storage config. That is harmless for default GCS ADC in the simplest case, but it drops configured backend settings like S3/R2 endpoints or GCS signing TTL/project. I?m checking whether the repo?s cloud-serving scope is intentionally GCS-only before I turn that into author feedback.

56. assistant (2026-07-04T07:49:25.932Z)

I?ve got enough signal for at least #465, and likely storage-config comments on #467/#469. Before posting anything, I?m checking existing PR comments/reviews so we don?t spam duplicates if another reviewer already called out the same points.

57. assistant (2026-07-04T07:49:45.529Z)

No duplicate comments are present on the PRs I checked. I?ll keep the comments tight: one red-

suite blocker on #465, and storage-config preservation where the new code would otherwise bypass configured backends.

58. assistant (2026-07-04T07:49:59.331Z)

#462 looks structurally straightforward so far: it adds the lazy Cloud Run SDK extra and wires the Dockerfile install string, with CI green. I?m doing one last sanity check that the declared package constraint is resolvable before leaving it uncommented.

59. assistant (2026-07-04T07:50:27.094Z)

I?m going to post three review comments now: #465?s full-suite failure, #467?s staging call dropping storage config, and #469?s mirror/download path dropping storage config. I?m not posting on #462, #464, or #468 unless a final pass turns up something concrete.

60. assistant (2026-07-04T07:50:37.336Z)

#465 now has the blocker comment with the exact failing test and expected reconciliation. I?m posting the storage-config comments next, because those are subtle enough that they deserve to be preserved on the PRs rather than buried in a summary.

61. assistant (2026-07-04T07:50:48.257Z)

#467 has the staging/config comment posted. One more similar-but-separate comment is going on #469 for both the mirror upload and signed-url lookup path.

62. assistant (2026-07-04T07:51:11.098Z)

All three actionable comments are posted. I?m doing a final sweep of the uncommented PRs and the exact CI state so the close-out is accurate and doesn?t overstate what was locally verified.

63. assistant (2026-07-04T07:51:35.649Z)

I?m going one layer deeper before the final summary: I?ll run the changed/targeted test files in detached temporary worktrees for the code PRs. That won?t satisfy the ?full local suite per PR? merge stamp, but it gives us useful local signal beyond CI and avoids disturbing the shared `master` checkout.

64. assistant (2026-07-04T07:52:57.942Z)

Targeted local tests matched the review picture: #462/#467/#468/#469 changed test files passed locally, and #465 reproduced the same failing hosted-mode test locally. The main checkout stayed clean; I?m just confirming no temporary worktrees were left behind.

65. assistant (2026-07-04T07:53:04.879Z)

No `bhi-review-pr*` temp worktrees were left behind. There are existing sibling worktrees for those feature branches already present, so I left them alone as likely owner/parallel-session state.

66. assistant (2026-07-04T07:53:28.269Z)

Reviewed all 6 open PRs and posted 3 comments:

- [#465 comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/465#issuecomment-4881192296): blocking, full suite fails because hosted-mode test still expects `gs://`/`s3://`/`gdrive://` to 400.
- [#467 comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/467#issuecomment-4881192642): staging upload calls `storage.put(...)` without passing active storage config.
- [#469 comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/469#issuecomment-4881193090): mirrored reel upload + signed URL lookup also drop active storage config.

No comments posted on #462, #464, or #468; I didn?t find actionable issues there.

Targeted local checks in temporary detached worktrees:
- #462: `tests/test_compute_backend.py` passed, 19 tests.
- #465: targeted check failed, reproducing the CI failure.
- #467: `tests/test_api_cloud_dispatch.py` passed, 25 tests.
- #468: `tests/test_drain_concurrency.py tests/test_config_deployment.py` passed, 39 tests.
- #469: `tests/test_upload_download.py` passed, 24 tests.

GitHub CI is green for #462/#464/#467/#468/#469 and red for #465. I did not run the full local suite separately for every PR, so I?m not stamping LGTM locally; merge-wise I?d hold #465/#467/#469 until the comments are addressed.

67. user (2026-07-04T07:53:28.902Z)

Once you are satisfied with the approach please drop an LGTM on the PR as a signal of handing off.

68. assistant (2026-07-04T07:53:47.892Z)

I?ll only use `LGTM` where it meets your pre-LGTM bar: findings resolved, full suite/coverage, lint, and type checks verified rather than inferred. I?ll re-fetch the PR state now, see which comments were addressed, and then run the required gates before posting any handoff signal.

69. assistant (2026-07-04T07:54:02.691Z)

Current heads haven?t changed since my comments, so #465/#467/#469 still have outstanding review feedback. I?m going to run the full local gates only for the PRs with no findings from my review (#462, #464, #468), then I?ll post LGTM there if they pass.

70. assistant (2026-07-04T07:54:12.165Z)

I?ll keep the verification isolated in temporary detached worktrees and remove them after each run. That lets us respect the shared-checkout rules while still satisfying the local verification gate for the PRs I?m prepared to hand off.

71. assistant (2026-07-04T08:04:52.377Z)

The LGTM gate passed locally for #462, #464, and #468: `ruff check .`, `mypy backend training`, and `run_all_tests.py` with the 70% coverage floor all exited 0 on each PR head. I?m going to leave LGTM handoff comments on those three only; the others still have unresolved comments or red CI.

72. assistant (2026-07-04T08:05:01.017Z)

LGTM handoff comments are now posted on #462, #464, and #468. I?m doing a quick cleanup check so the shared repo state is still clean before I wrap.

73. assistant (2026-07-04T08:05:15.172Z)

Posted LGTM handoff comments on the PRs that met the full local gate:

- [#462 LGTM](https://github.com/Khelsutra/badminton-highlight-indexer/pull/462#issuecomment-4881229777)
- [#464 LGTM](https://github.com/Khelsutra/badminton-highlight-indexer/pull/464#issuecomment-4881229780)
- [#468 LGTM](https://github.com/Khelsutra/badminton-highlight-indexer/pull/468#issuecomment-4881229785)

For each, I verified locally on the PR head: `ruff check .`, `mypy backend training`, and `run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`.

I did not LGTM #465/#467/#469 because my review comments are still outstanding on the remote PRs I checked. The main checkout is clean; temporary LGTM worktrees were removed.

74. user (2026-07-04T08:06:26.310Z)

Did you not do a logic check of [Khelsutra/badminton-highlight-indexer#468](https://github.com/Khelsutra/badminton-highlight-indexer/pull/468)

75. assistant (2026-07-04T08:06:39.467Z)

I did inspect #468?s diff and specifically checked the drain-concurrency startup wiring, but you?re right to challenge the depth before an LGTM. I?m going to re-open that PR now as a logic review, re-fetch the latest head, and verify whether there?s a behavioral hole that tests/CI didn?t catch.

76. assistant (2026-07-04T08:06:51.327Z)

The PR head is still `d56237f`; it has not changed since my LGTM comment. I?m reading the relevant worker loop, DB claim semantics, and server startup paths together now, because this is exactly the kind of change where ?tests green? can miss a queueing/concurrency invariant.

77. assistant (2026-07-04T08:07:10.024Z)

A more precise logic issue is emerging: the PR scales the executor only from the server?s `deployment.compute_target == "cloud_run"`, but the code also has a per-request `compute: "cloud"` path that can dispatch Cloud Run when cloud resources are configured even if the server default target is not `cloud_run`. I?m checking whether that path is intentional/supported and whether this PR leaves it serialized.

78. assistant (2026-07-04T08:07:26.326Z)

The important nuance: #468 probably does help the observed ?second request arrives while first cloud poll is running? case, because each POST submits another drain task and the larger executor can run those tasks concurrently. But I?m not yet convinced it covers all cloud-capable entry paths, especially the local-default `compute="cloud"` override and app-factory/TestClient-style startup, so I?m going to validate those assumptions rather than wave it through retroactively.

79. assistant (2026-07-04T08:08:01.950Z)

I found a real logic gap I should have called out before LGTM: the PR scales the global executor, not ?remote detect jobs? specifically. That means on a `cloud_run` server, compile jobs and `compute="local"` jobs can also run in parallel, while a local-default-but-cloud-capable server using per-request `compute="cloud"` remains serial. I?m going to post a correction on the PR and withdraw the LGTM signal.

80. assistant (2026-07-04T08:08:18.190Z)

The correction is now posted on #468. I?m checking the local checkout one last time, then I?ll summarize plainly what happened.

81. assistant (2026-07-04T08:08:23.764Z)

You?re right to call that out. I did a logic pass, but it was too shallow before I posted LGTM.

I re-reviewed #468 and found a real gap: it scales the global async executor, not specifically remote Cloud Run detect jobs. I posted a correction and withdrew my LGTM here: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/468#issuecomment-4881239809).

Main issue: compile/local override jobs can now run concurrently on `cloud_run` servers, while per-request `compute="cloud"` jobs on a local-default-but-cloud-capable server still remain serialized.

82. user (2026-07-04T08:28:20.944Z)

can ptal at #468 again?

83. assistant (2026-07-04T08:28:36.742Z)

I?ll recheck #468 from the current remote head, compare it against the issue I withdrew LGTM for, and only re-LGTM if the logic and verification both hold. First I?m refreshing the repo and PR metadata so we?re not reviewing a stale branch.

84. assistant (2026-07-04T08:28:50.376Z)

The author has pushed a new head (`8534077`) with a response to the exact issue: a local compute lane semaphore plus updated worker-count detection for cloud overrides. I?m going to inspect that new diff carefully now, then wait for/verify the full suite before deciding whether to re-LGTM.

85. assistant (2026-07-04T08:29:07.152Z)

The first `git pull` hit a local branch-config oddity (?Cannot fast-forward to multiple branches?), but the PR ref fetch succeeded and I have the current #468 patch. I?ll repair freshness with an explicit `origin master` fast-forward and continue read-only against the fetched PR head.

86. assistant (2026-07-04T08:29:46.661Z)

One deeper concurrency invariant I?m checking now is the DB claim loop itself. With multiple drain tasks, an atomic compare-and-set prevents double-claiming, but if a worker loses the race for the first queued row and exits instead of retrying, concurrency can silently collapse back to one worker. I?m going to reproduce or falsify that against the PR head.

87. assistant (2026-07-04T08:30:36.682Z)

I?ll re-open #468 from the latest head and specifically redo the logic pass around the new concurrency path, not just skim the patch. I already have the prior concern in mind, so I?m going to verify whether the replacement approach actually preserves parallel dispatch under contention before deciding whether it?s merge-ready.

88. assistant (2026-07-04T08:30:51.634Z)

I?m applying the review workflow again here: freshen the local repo first, then inspect the actual PR head. After that I?ll only comment if there?s something actionable rather than generating noise.

89. assistant (2026-07-04T08:31:02.493Z)

The repo is fresh against `origin/master`. I?m pulling the PR metadata and diff now so the review is anchored to the current commit, not the earlier version I already challenged.

90. assistant (2026-07-04T08:31:15.738Z)

The new CI run is green and the PR is clean to merge structurally, so the remaining question is purely behavior. I?m checking the job-claiming/drain loop now because this PR only works if parallel workers reliably claim distinct queued jobs under contention.

91. assistant (2026-07-04T08:31:34.164Z)

There?s a subtle race hiding below the PR diff: the database claim operation was safe for single-worker draining, but parallel drain workers change its assumptions. I?m reading that path against the PR head before I decide whether this is comment-worthy or just theoretical.

92. assistant (2026-07-04T08:31:47.294Z)

This is looking real rather than hypothetical: `claim_due_job()` returns `None` on a lost compare-and-set race, and the drain interprets `None` as ?queue empty.? I?m going to reproduce

it with concurrent claimers so the PR comment can be evidence-based and easy for the author to fix.

93. assistant (2026-07-04T08:31:53.696Z)

I?m using a small local reproducer rather than guessing from code shape. The goal is simple: seed N queued jobs, have N drain workers call `claim_due_job()` at once, and see whether they claim N distinct jobs or falsely stop on `None`.

94. assistant (2026-07-04T08:32:15.957Z)

The reproducer confirms the bug: on the current code path, 8 concurrent claim attempts over 8 queued jobs only claimed 2 in one run because the other workers hit the lost-race `None` path and exited. I?m going to leave that as a focused PR comment; this is exactly the kind of thing that should be fixed before we stamp it.

95. assistant (2026-07-04T08:32:32.543Z)

I posted the blocker comment on the PR with the reproducer output and a concrete fix shape. I?m doing a quick sanity check of local state and PR status now so I can give you a clean merge-readiness answer rather than just ?commented.?

96. assistant (2026-07-04T08:32:42.302Z)

Rechecked #468 and found one more logic blocker. I posted it here: [PR comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/468#issuecomment-4881310273).

The issue is a race in `claim_due_job()`: parallel drain workers can all select the same queued row, one wins, the others return `None`, and `_drain_due_jobs()` treats that as ?queue empty? even while other jobs remain queued. I reproduced it locally with 8 queued jobs and 8 concurrent claimers; only 2 were claimed.

CI is green and the PR is structurally clean, but I would not merge yet and did not leave LGTM. The fix should make lost claim races retry or otherwise distinguish ?lost race? from ?empty queue,? with a concurrent regression test.

97. user (2026-07-04T08:33:33.176Z)

cool! Can you review the other remaining ones in the meanwhile?

98. assistant (2026-07-04T08:33:46.036Z)

I?ll review the remaining open PRs in `Khelsutra/badminton-highlight-indexer` while #468 is being fixed, excluding #468 unless it changes again. I?m using the code-review workflow again, starting with a fresh pull and the current open-PR list so we don?t review stale heads.

99. assistant (2026-07-04T08:33:54.099Z)

I?m loading the review skill instructions and repo state first; then I?ll fan out across the PRs. Tiny reviewer goblin hat is on, but we?ll keep it useful and evidence-based.

100. assistant (2026-07-04T08:34:00.398Z)

The local checkout is clean and current with `origin/master`. Next I?m listing the open PRs and their CI status, then I?ll work through the remaining ones one by one.

101. assistant (2026-07-04T08:34:16.062Z)

Open PRs besides #468 are #465, #467, and #469, all with green CI and clean merge state at first glance. I?m pulling their diffs and existing comments together now so I can tell whether prior feedback was actually fixed rather than just CI-green.

102. assistant (2026-07-04T08:34:24.633Z)

Prior comments on #465, #467, and #469 have author replies and fresh green CI, so I?m now checking the actual diffs for the fix shape and any new edge cases. The goal is either a crisp follow-up comment or a verified LGTM, not a vibes-based approval.

103. assistant (2026-07-04T08:34:34.359Z)

The first pass shows the earlier storage-config comments were addressed in #467 and #469, and #465 now gates URI access by configured `storage.input_root` instead of allowing arbitrary buckets. I?m fetching the PR heads so I can inspect the exact branch versions and run targeted checks cleanly.

104. assistant (2026-07-04T08:34:53.613Z)

I?m looking a little below the changed lines now: for these cloud-serving fixes, the important mistakes tend to be ?helper looks correct but caller gives it the wrong config? or ?DB state points at the old ephemeral path.? That?s where the dragon teeth usually are.

105. assistant (2026-07-04T08:35:05.984Z)

The changed behavior mostly hangs together. I?ve got one deeper thing to verify for #469: the new storage helpers rely on every backend having an `exists`/`signed_url` contract, so I?m checking interface/default behavior to avoid approving something that works only for the mocked tests.

106. assistant (2026-07-04T08:35:42.219Z)

I found a likely actionable issue in #469: the config dict is passed through, but the signed URL helper?s default `ttl=3600` may still override the configured backend TTL. I?m going to reproduce that locally before commenting, and I?m also checking #467?s mutation/cleanup path because it rewrites `req.video_path` after staging.

107. assistant (2026-07-04T08:36:08.353Z)

A second concrete edge case surfaced in #467: the staging check keys off `storage.input_backend`, but an explicit `storage.input_root="gs://..."` is already enough for the storage layer to know the backend. If someone only sets the URI root and leaves `input_backend` at its default, staging still fails. I?m checking whether repo presets always set both or whether this needs a PR comment.

108. assistant (2026-07-04T08:36:40.086Z)

I?ve got two likely follow-up findings to validate: #467?s explicit-URI `input_root` case and #469?s configured signed URL TTL. I?m going to run small branch-head repros for both so the comments are grounded in actual behavior, not reviewer chessboard imagination.

109. assistant (2026-07-04T08:37:21.905Z)

I?m running those repros now. If they reproduce, I?ll post narrow comments on the affected PRs; if they don?t, I?ll fold the notes back into the ?looks good? bucket and move on.

110. assistant (2026-07-04T08:37:45.636Z)

Both repros landed. #467 rejects a valid explicit `gs://` input root when `input_backend` is left at its default, and #469 ignores configured `signed_url_ttl` by passing the helper?s hardcoded default down to the backend. I?m posting those as focused comments now.

111. assistant (2026-07-04T08:37:54.393Z)

Comments are posted on #467 and #469. #465 is the one that still looks potentially mergeable, so I?m putting it through the local pre-LGTM gate now rather than approving from CI alone.

112. assistant (2026-07-04T08:38:02.778Z)

For #465, I?m going to verify the repo?s stated gates: full suite with coverage floor, `ruff`, and `mypy`. If those pass and coverage is not below the base floor, I?ll leave the LGTM; otherwise I?ll report exactly where it stops.

113. assistant (2026-07-04T08:38:13.349Z)

The #465 worktree is checked out at `cd4bc73`. I?m kicking off the full test+coverage gate alongside lint/type checks now; this is the part where we let the boring machinery earn its keep.

114. assistant (2026-07-04T08:40:55.966Z)

#465?s PR-head gates passed locally: full suite `1634 passed`, coverage `85.40%` over the `70%` floor, `ruff` clean, and `mypy` clean. I?m doing the last bit of the LGTM gate now: compare coverage against `origin/master` locally, because ?above the floor? and ?not regressed? are separate promises.

115. assistant (2026-07-04T08:44:46.443Z)

#465 passed the floor but did not pass the stricter ?no coverage regression? gate: PR head reported `85.40%`, while local `origin/master` reported `85.49%`. I?m checking the missing-line detail now so the feedback can be precise instead of just ?coverage number went down.?

116. assistant (2026-07-04T08:45:11.344Z)

One wrinkle: the absolute coverage comparison is a small drop, but it appears influenced by unrelated coverage variance/skips rather than uncovered new lines. I?m checking the commit relationship and changed-file coverage before deciding whether to block #465 or simply avoid LGTM until the author/CI coverage policy clarifies it.

117. assistant (2026-07-04T08:46:03.470Z)

Important correction on that coverage check: #465?s branch is behind two already-merged master commits, so comparing raw PR head coverage to `origin/master` is not apples-to-apples. The real merge diff is still just the three intended files, so I?m testing the GitHub merge ref now before deciding on LGTM.

118. assistant (2026-07-04T08:46:19.511Z)

The merge ref is checked out now, which is the right thing to verify for this slightly-behind branch. I?m rerunning the full gate there; if it?s clean and coverage is at least master?s local `85.49%`, #465 can get a proper LGTM.

119. assistant (2026-07-04T08:49:09.721Z)

The GitHub merge ref for #465 is stale too: it merges into `a5d624`, not today?s `origin/master`. Since master moved underneath this PR, I?m going to create a local current-master merge and test that actual result; otherwise an LGTM would be built on old ground.

120. assistant (2026-07-04T08:49:22.315Z)

The current-master merge applies cleanly and only touches the intended three files. I?m running the gate one final time on that actual merge result; this should settle #465 either way.

121. assistant (2026-07-04T08:52:37.982Z)

The current-merge result is clean on tests/lint/type, but the coverage delta is still polluted by running master in the main checkout while PR runs in a fresh temp worktree. I?m doing a like-for-like master worktree coverage run so we don?t accidentally penalize #465 for local untracked data or environment differences.

122. assistant (2026-07-04T08:52:42.734Z)

This is the last expensive check. Once this base worktree run finishes, I?ll either LGTM #465 or leave it unapproved with the exact remaining reason.

123. assistant (2026-07-04T08:55:35.361Z)

The like-for-like comparison clears #465: base worktree coverage was `85.39%`, current-master + #465 merge was `85.40%`, so coverage does not regress. I?m posting the LGTM on #465 with the exact local verification, then I?ll clean up the temporary worktrees.

124. assistant (2026-07-04T08:55:45.080Z)

#465 has its LGTM now. I?m removing the temporary review worktrees and then I?ll give you the final state of the remaining PRs.

125. assistant (2026-07-04T08:55:58.595Z)

The temp directories were removed, but Git left behind some locked/stale worktree metadata with permission-denied warnings. I?m checking the worktree registry and will prune stale entries if Git sees them as gone.

126. assistant (2026-07-04T08:56:04.223Z)

Git?s worktree list no longer includes my temp worktrees, and the main checkout is clean. I?m checking whether those metadata dirs actually remain; if they do, I?ll leave them alone rather than force-delete Git internals unnecessarily.

127. assistant (2026-07-04T08:56:09.501Z)

The stale metadata directories are still present even though Git no longer reports those worktrees. I?ll try a non-destructive `git worktree prune` pass; if Windows keeps the metadata locked, it?s harmless enough to note and move on.

128. assistant (2026-07-04T08:56:25.906Z)

`git worktree prune` confirms those are stale metadata entries, but Windows is denying deletion of the `.git/worktrees/*` folders. The actual temp working directories are gone and `git worktree list` is clean for my review work; I?ll leave the locked metadata alone to avoid poking Git internals.

129. assistant (2026-07-04T08:56:52.683Z)

Reviewed the other open PRs.

Results
- #465: LGTM posted: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/465#issuecomment-4881370728). Verified on a current-`origin/master` local merge: `1634 passed, 10 skipped`, coverage `85.40%` vs base worktree `85.39%`, `ruff` clean, `mypy` clean.
- #467: Not merge-ready. Posted a blocker: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/467#issuecomment-4881323347). The staging gate rejects valid `input_root="gs://..."` configs when `input_backend` is left default.
- #469: Not merge-ready. Posted a blocker: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/469#issuecomment-4881323349). The new signed URL helper ignores configured `signed_url_ttl`.

Main checkout is clean. #468 still has the earlier claim-race blocker open, so currently only #465 has my LGTM.

130. user (2026-07-04T09:23:25.791Z)

hello... The author has fixed all the PRs. Can you take another thorough pass through all of

them and LGTM ones you deem merge ready

131. assistant (2026-07-04T09:23:45.349Z)

I?ll re-review the full current open PR set from fresh heads, including the ones we previously blocked, and only leave LGTM where the logic pass and the local verification gate both clear. First I?m refreshing the repo and loading the review workflow so we?re anchored to current remote state.

132. assistant (2026-07-04T09:23:50.438Z)

The main checkout is clean and current. I?m listing the open PRs and current CI/head SHAs next, then I?ll inspect each updated diff plus the specific prior blockers before running the merge-gate checks.

133. assistant (2026-07-04T09:24:09.571Z)

There are five open PRs now: #467, #468, #469, plus new #470 and #474. #467/#468/#469 have fresh heads after the fixes; #474 still has CI full-suite in progress, so I?ll review it but won?t LGTM it until that gate and the local gate are both settled.

134. assistant (2026-07-04T09:24:21.938Z)

I?m pulling metadata, comments, and diffs for the five heads now. The priority is to re-test the exact blockers we had: claim-race retry in #468, explicit `input_root` staging in #467, signed URL TTL in #469, then do fresh logic passes on #470/#474.

135. assistant (2026-07-04T09:24:38.730Z)

The updated author replies line up with the earlier blockers, and CI has gone green for #474 now too. I?m fetching all current heads and reading the actual diffs so I can verify the code, not just the comments.

136. assistant (2026-07-04T09:24:51.648Z)

All five PR refs are fresh locally. I?m starting with diff stats and changed-file lists to spot stacking/overlap, then I?ll drill into the logic-heavy files.

137. assistant (2026-07-04T09:25:03.856Z)

The PRs are independent and small enough to review directly. I?m drilling into the implementation files and tests now, especially the new Cloud Run manifest generator because deploy YAML tends to hide sharp edges in quoting, secrets, and runtime flags.

138. assistant (2026-07-04T09:25:43.862Z)

The earlier blockers appear fixed in code: #467 now classifies `input_root` by URI scheme, #468 retries lost CAS races, and #469 lets backend TTL config flow through. I?m doing one extra pass on the new service-manifest PR because generated deploy config can be correct syntactically while still creating a bad runtime posture.

139. assistant (2026-07-04T09:26:59.269Z)

I?m picking this back up as a proper merge-gate pass, not just a surface scan. I?ll refresh the repo, inspect each currently open PR against the latest `master`, rerun the local verification gates for merge-ready candidates, then leave either blocker comments or an LGTM with exactly what was verified.

140. assistant (2026-07-04T09:27:08.944Z)

I?m using the code-review skill here because this is exactly the ?review before merge? path, and I?ll keep the LGTM threshold aligned with your pre-merge gate rather than treating CI as enough.

141. assistant (2026-07-04T09:27:27.740Z)

I?m also checking the project handoff before digging deeper, so this pass uses the latest repo context rather than only the PR descriptions. After that I?ll work PR-by-PR and only use ?merge ready? where the local gate backs it up.

142. assistant (2026-07-04T09:27:39.365Z)

The open set is still five PRs: #467, #468, #469, #470, and #474, all cleanly mergeable with green GitHub CI. I?m fetching their exact heads now and will review the actual deltas, especially the earlier logic fixes rather than trusting the author notes.

143. assistant (2026-07-04T09:27:54.902Z)

The handoff is old A7/A9 context rather than this PR batch, so I?m treating it as background only. The review focus is the cloud-serving series and whether the fixes now close the earlier correctness gaps.

144. assistant (2026-07-04T09:28:00.723Z)

#467, #468, and #469 are the ones with prior correctness concerns, so I?m starting there: upload staging semantics, drain concurrency, and signed URL TTL propagation. #470 and #474 are deployment/manifest changes and will get the same gate after the logic pass.

145. assistant (2026-07-04T09:28:22.134Z)

The first fix (#467) now classifies the configured `input_root` by URI scheme instead of relying on `input_backend`, which addresses the previous default-`local` false rejection. I?m checking #468?s actual race fix now because that one is subtle: `None` from a claimant must mean the queue is truly empty, not ?lost the compare-and-swap.?

146. assistant (2026-07-04T09:28:35.286Z)

#468?s CAS loop now retries after a lost race, which is the core fix I wanted. I?m sanity-checking the concurrency expansion around it too, especially that local CPU/GPU and compile paths stay serialized while remote dispatch/polling can overlap.

147. assistant (2026-07-04T09:28:50.647Z)

#469 now moves the default TTL responsibility to the backend instead of the module helper, so configured GCS/S3 TTLs can actually survive. I?m checking the storage API shape and the `/api/highlights` fallback path to make sure the signed redirect doesn?t break local downloads.

148. assistant (2026-07-04T09:29:00.154Z)

#470 is deliberately small and looks like the right deployment fix: the reused Cloud Run image now installs the `auth` extra, guarded by the Dockerfile structural test. I?m moving to #474?s manifest generator, which is the newest surface area and needs a little more deployment-shape scrutiny.

149. assistant (2026-07-04T09:29:41.492Z)

One deployment detail I?m checking on #474: the generated config writes uploads under `RALLY_APP_HOME`, while the app has a few historical relative paths. If those diverge, uploads or local fallback downloads could reject themselves, so I?m tracing the path resolver instead of eyeballing the YAML.

150. assistant (2026-07-04T09:30:08.075Z)

The manifest generator?s wrapper/config path lines up with the hosted upload jail: uploads resolve under `/tmp/apphome/output/uploads`, and bucket inputs are constrained to the generated `input_root`. I don?t see a logic blocker in #474 yet, so I?m moving into the local verification gate.

151. assistant (2026-07-04T09:30:32.378Z)

All five still look like candidates after the logic pass, so I?m setting up isolated detached worktrees for local gate runs. That lets us test each PR exactly as fetched without disturbing the main checkout or any sibling session?s branches.

152. assistant (2026-07-04T09:30:38.629Z)

The verification workspace is `C:\Users\avidu\AppData\Local\Temp\codex-pr-review-20260704-150034`. I?m creating a base worktree plus one detached worktree per PR so coverage comparisons are apples-to-apples.

153. assistant (2026-07-04T09:31:18.944Z)

I?m going to run the local gate as: `ruff check .`, `mypy backend training`, then `python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`. I?ll run base plus the first two PRs in parallel, then the remaining three, and compare coverage back to base.

154. assistant (2026-07-04T09:31:38.171Z)

The first gate batch has been launched in hidden PowerShell processes; I?m checking the logs/PIDs now so I can monitor progress rather than waiting blindly.

155. assistant (2026-07-04T09:31:45.651Z)

The logs are buffered, so they?ll probably fill only when each process exits. That?s okay; I?m monitoring the worker processes and will read the gate logs as soon as each finishes.

156. assistant (2026-07-04T09:32:42.517Z)

The first batch is still running; the shell wrappers are mostly waiting on child Python processes now. I?m checking the Python children so I can tell whether they?re in test execution versus stuck before output.

157. assistant (2026-07-04T09:32:46.921Z)

The first batch passed `ruff` and `mypy` far enough to reach `run_all_tests.py`; all three are currently in pytest with coverage. That?s the slow but meaningful part.

158. assistant (2026-07-04T09:33:53.678Z)

Still in the coverage suite; no failures have surfaced yet because the logs are held until process exit. I?m letting this finish rather than shortcutting, since this is the verification line that makes the LGTM meaningful.

159. assistant (2026-07-04T09:35:15.085Z)

This batch is taking longer than CI, likely because three full coverage suites are sharing the same machine. The processes are alive rather than wedged, so I?m giving them a bit more time before deciding whether to reduce concurrency.

160. assistant (2026-07-04T09:37:09.692Z)

The pytest children have exited now; I?m reading the gate logs for base, #467, and #468 to confirm exit codes, coverage, and whether anything was skipped unexpectedly.

161. assistant (2026-07-04T09:37:21.819Z)

The first batch?s PR-head gates are green, but I caught an important review hygiene issue before posting anything: those detached heads are not the same as the final merge tree against current `master`. I?m going to rerun the gate on local merge worktrees, which is the right standard for ?merge ready.?

162. assistant (2026-07-04T09:37:39.185Z)

The PR-head checks found no failures, and now I?m switching the gate to the true merge candidate shape: `origin/master` plus each PR merged with `--no-commit`. That catches cross-PR/base interactions and avoids giving a stamp based on stale branch contents.

163. assistant (2026-07-04T09:37:59.771Z)

All five merge worktrees merged cleanly with current `origin/master`. I?m starting the first merge-tree gate batch now (#467, #468, #469), which includes the three PRs with earlier logic blockers.

164. assistant (2026-07-04T09:38:21.158Z)

The first merge-tree gate batch is launched; output is buffered again, so I?ll monitor the spawned Python processes and read the logs once they flush.

165. assistant (2026-07-04T09:38:25.615Z)

This merge batch has cleared ruff and is in mypy now. Once it gets into the suite, I?ll let it run to completion and then compare each coverage number to the base 85.40%.

166. assistant (2026-07-04T09:39:20.186Z)

Mypy finished for #467/#468/#469 and all three are now running the full coverage suite. This is the slow part, but it?s exactly where regressions like the earlier drain race would show up.

167. assistant (2026-07-04T09:41:26.151Z)

The three suites are still alive after two minutes, likely just competing for CPU. I?ll keep them together for this batch, then run #470/#474 as a smaller second batch so we don?t stretch the machine too much.

168. assistant (2026-07-04T09:43:42.244Z)

The first merge-tree batch has finished. I?m reading the logs now for exit codes, coverage, and the previous blocker tests before I move on to #470/#474.

169. assistant (2026-07-04T09:44:01.270Z)

The first merge-tree batch passed ruff, mypy, and the suite, but I?m pausing on the coverage comparison because two PRs are a hair below the freshly measured base percentage. Your gate says ?no regression,? so I?m checking the exact coverage totals before deciding whether that is a blocker or just rounded noise.

170. assistant (2026-07-04T09:44:20.931Z)

Exact coverage confirms the subtle issue: #468 is above base, while #467 and #469 are slightly below the base total despite passing the 70% floor. I?ll finish #470/#474 before posting anything, then I?ll LGTM only the PRs that clear every part of the gate.

171. assistant (2026-07-04T09:44:39.151Z)

#470 and #474 are now running through the same merge-tree gate. Since this batch is only two suites, it should be a bit friendlier to the machine.

172. user (2026-07-04T09:44:50.648Z)

<turn_aborted>
The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.
</turn_aborted>

173. user (2026-07-04T09:44:54.758Z)

I guess I should ask you to do logic only review or overall review .. I wanted this one to be a logic only review, you can omit the mypy or ruff etc. checks and focus on the logic itself

174. assistant (2026-07-04T09:45:08.769Z)

That makes sense. I?m going to stop any still-running local gate processes from the aborted run, then switch this to a logic-only pass and make any PR comments explicitly as logic-review comments rather than ?full local merge gate verified.?

175. assistant (2026-07-04T09:45:19.424Z)

There are two leftover suite runners from the interrupted #470/#474 gate. I?m stopping those now so we don?t burn CPU on checks you explicitly don?t need for this review.

176. assistant (2026-07-04T09:45:44.053Z)

I?ll still be disciplined about the scope: I?m not going to call ?full gate verified? on these. I?ll post LGTM only as ?logic-review LGTM? where the implementation path looks sound, and any blocker would be a logic/correctness blocker rather than lint/type/coverage.

177. assistant (2026-07-04T09:45:48.228Z)

The manifest?s ?no `CLOUD_RUN_JOB` env? assumption checks out: the dispatch code?s default job name is `rally-worker`, so avoiding the reserved env name doesn?t leave the service unable to dispatch.

178. assistant (2026-07-04T09:46:30.705Z)

Logic pass result: I don?t see remaining correctness blockers in #467/#468/#469/#470/#474. The only nuance is #474?s manifest assumes the image includes the auth extra, so I?ll call that out as a merge-order/deploy note rather than an author fix.

179. assistant (2026-07-04T09:46:38.536Z)

No leftover gate processes are running now. I?m doing one quick PR-list refresh, then I?ll post the logic-only handoff comments/LGTMs on the still-open PRs.

180. assistant (2026-07-04T09:47:02.312Z)

I?m posting one concise logic-review LGTM per PR, with the scope spelled out so future readers don?t mistake it for a fresh local lint/type/coverage certification.

181. assistant (2026-07-04T09:47:29.448Z)

The PR comments are posted. I?m cleaning up the temporary detached/merge worktrees I created for the earlier gate attempt so the workspace doesn?t accumulate review scratchpads.

182. assistant (2026-07-04T09:47:46.239Z)

Posted logic-review LGTMs on all remaining PRs:

- #467: https://github.com/Khelsutra/badminton-highlight-indexer/pull/467#issuecomment-4881514264
- #468: https://github.com/Khelsutra/badminton-highlight-indexer/pull/468#issuecomment-4881514309
- #469: https://github.com/Khelsutra/badminton-highlight-indexer/pull/469#issuecomment-4881514356
- #470: https://github.com/Khelsutra/badminton-highlight-indexer/pull/470#issuecomment-4881514415
- #474: https://github.com/Khelsutra/badminton-highlight-indexer/pull/474#issuecomment-4881514470

No remaining logic blockers found. I scoped the comments explicitly as logic-only and noted the #474 deploy-order caveat: use it after #470 or with an image rebuilt to include the `auth` extra. I also stopped the leftover local gate runners and cleaned up the temp worktrees.